

21. 合并两个有序链表

1. 方法

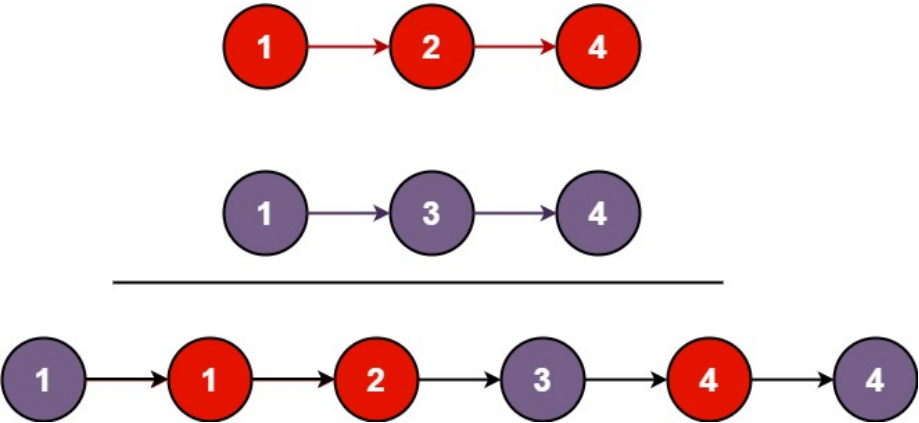
合并两个有序链表

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Easy (67.62%)	3768	0	-	-	0

- Tags
- Companies

将两个升序链表合并为一个新的 升序 链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

示例 1:



输入: l1 = [1,2,4], l2 = [1,3,4]
输出: [1,1,2,3,4,4]

示例 2:

输入: l1 = [], l2 = []
输出: []

示例 3:

输入: l1 = [], l2 = [0]
输出: [0]

提示:

- 两个链表的节点数目范围是 [0, 50]
- 100 <= Node.val <= 100

- 11 和 12 均按 非递减顺序 排列

Discussion | Solution

2. 方法

LeetCode 21: 合并两个有序链表（详细解析）

1. 问题理解

目标：将两个升序排列的链表合并为一个新的升序链表。

输入：两个已排序的单链表的头节点 `list1` 和 `list2`

输出：合并后的新链表的头节点

示例：

- 输入：`list1 = [1,2,4]`, `list2 = [1,3,4]`
- 输出：`[1,1,2,3,4,4]`

2. 基本思路

我们需要创建一个新的链表，这个新链表的节点来自于 `list1` 和 `list2`，并且保持升序排列。

我们可以使用两种主要方法：

1. 迭代法 - 使用循环，比较两个链表的当前节点，将较小的节点添加到新链表中。
2. 递归法 - 比较两个链表的头节点，确定新链表的头，然后递归地合并剩余部分。

3. 解法一：迭代法（推荐）

思路

核心思想：维护一个指向新链表尾部的指针 `current`，同时遍历 `list1` 和 `list2`。在每一步中，比较两个链表当前节点的值，将较小的节点连接到 `current` 的后面，并移动相应链表的指针和 `current` 指针。

辅助节点：为了方便处理空链表和新链表的头节点，通常会创建一个哑节点（**dummy node**）作为新链表的伪头节点。最终返回哑节点的下一个节点即可。

步骤

1. 创建一个哑节点 `dummy` 和一个指向它的指针 `current`。
2. 使用两个指针 `p1` 和 `p2` 分别指向 `list1` 和 `list2` 的头节点。
3. 当 `p1` 和 `p2` 都不为空时，进行比较：

- 如果 `p1.val <= p2.val`, 将 `p1` 连接到 `current.next`, 然后移动 `p1` 和 `current`。
 - 否则 (`p1.val > p2.val`), 将 `p2` 连接到 `current.next`, 然后移动 `p2` 和 `current`。
4. 循环结束后, 可能有一个链表还有剩余节点 (因为另一个链表已经遍历完了)。将这个剩余的链表直接连接到 `current.next`。
 5. 返回 `dummy.next` 作为合并后链表的头节点。

代码实现

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    /**
     * 合并两个有序链表 - 迭代法
     * @param list1 第一个有序链表
     * @param list2 第二个有序链表
     * @return 合并后的有序链表
     */
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        // 创建哑节点, 简化边界处理
        ListNode dummy = new ListNode(-1);
        // current 指针用于构建新链表
        ListNode current = dummy;

        // 使用 p1 和 p2 遍历 list1 和 list2
        ListNode p1 = list1;
        ListNode p2 = list2;

        // 当两个链表都还有节点时
        while (p1 != null && p2 != null) {
            // 比较 p1 和 p2 指向的节点值
            if (p1.val <= p2.val) {
                // 将较小的节点 (p1) 连接到新链表
                current.next = p1;
                // 移动 p1 指针
                p1 = p1.next;
            } else {

```

```

        // 将较小的节点 (p2) 连接到新链表
        current.next = p2;
        // 移动 p2 指针
        p2 = p2.next;
    }
    // 移动 current 指针到新链表的末尾
    current = current.next;
}

// 处理剩余的节点
// 如果 list1 还有剩余节点, 直接连接
if (p1 != null) {
    current.next = p1;
}
// 如果 list2 还有剩余节点, 直接连接
else if (p2 != null) {
    current.next = p2;
}

// 返回哑节点的下一个节点, 即合并后链表的头节点
return dummy.next;
}
}

```

复杂度分析

- 时间复杂度： $O(m + n)$ ，其中 m 和 n 分别是两个链表的长度。需要遍历两个链表的所有节点一次。
- 空间复杂度： $O(1)$ ，只使用了常数个额外变量（指针），没有使用与输入规模相关的额外空间。

4. 解法二：递归法

思路

核心思想：比较两个链表的头节点，确定合并后链表的头节点，然后递归地合并剩余的部分。

步骤

1. 基本情况：如果 `list1` 为空，返回 `list2`；如果 `list2` 为空，返回 `list1`。
2. 递归步骤：
 - 比较 `list1.val` 和 `list2.val`。
 - 如果 `list1.val <= list2.val`：
 - 合并后链表的头是 `list1`。
 - 递归调用 `mergeTwoLists(list1.next, list2)` 来合并 `list1` 的剩余部分和整个 `list2`。

- 将递归结果连接到 `list1.next`。
- 返回 `list1`。
- 否则 (`list1.val > list2.val`):
 - 合并后链表的头是 `list2`。
 - 递归调用 `mergeTwoLists(list1, list2.next)` 来合并整个 `list1` 和 `list2` 的剩余部分。
 - 将递归结果连接到 `list2.next`。
 - 返回 `list2`。

代码实现

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    /**
     * 合并两个有序链表 - 递归法
     * @param list1 第一个有序链表
     * @param list2 第二个有序链表
     * @return 合并后的有序链表
     */
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        // 基本情况: 如果一个链表为空, 返回另一个链表
        if (list1 == null) {
            return list2;
        }
        if (list2 == null) {
            return list1;
        }

        // 递归步骤: 比较头节点, 确定新头节点, 并递归合并剩余部分
        if (list1.val <= list2.val) {
            // list1 的头节点是新头节点
            // 递归合并 list1.next 和 list2
            list1.next = mergeTwoLists(list1.next, list2);
            // 返回新头节点 list1
            return list1;
        } else {
            // list2 的头节点是新头节点

```

```

        // 递归合并 list1 和 list2.next
        list2.next = mergeTwoLists(list1, list2.next);
        // 返回新头节点 list2
        return list2;
    }
}
}

```

复杂度分析

- 时间复杂度： $O(m + n)$ ，每个节点只会被访问一次。
- 空间复杂度： $O(m + n)$ ，由于递归调用使用了调用栈，最大调用深度为 $m + n$ 。

5. 如何在面试中讲解

1. 确认问题理解：
 - “我们需要合并两个已排序的链表，生成一个新的已排序链表。”
2. 提出两种方法：
 - “这个问题可以用迭代或递归两种方法解决。迭代法通常空间效率更高。”
3. 讲解迭代方法（推荐）：
 - “我们可以创建一个哑节点 `dummy` 来简化头节点的处理。”
 - “使用一个 `current` 指针指向新链表的尾部，同时用 `p1` 和 `p2` 遍历两个输入链表。”
 - “在循环中，比较 `p1` 和 `p2` 的值，将较小的节点连接到 `current.next`，并移动相应的指针。”
 - “循环结束后，将未遍历完的链表剩余部分连接到 `current.next`。”
 - “最后返回 `dummy.next`。”
4. 讲解递归方法：
 - “递归方法思路是：比较两个链表的头节点，确定新链表的头，然后递归地合并剩余部分。”
 - “基本情况是当一个链表为空时，返回另一个链表。”
 - “递归步骤中，如果 `list1.val` 较小，则 `list1` 是头，其 `next` 指向 `mergeTwoLists(list1.next, list2)` 的结果；反之亦然。”
5. 比较两种方法：
 - “迭代法空间复杂度为 $O(1)$ ，递归法为 $O(m+n)$ （因为递归栈）。”
 - “两者时间复杂度都是 $O(m+n)$ 。”
 - “在空间有限制或链表很长的情况下，迭代法通常是更好的选择。”

迭代法是解决这个问题的标准且高效的方法。

找到具有 1 个许可证类型的类似代码